

Lab 3

Nicholas Storti

July 2, 2026

1 Output

```
Setting up the problem...0.053358 s
A: 2048 x 2048
B: 2048 x 2048
C: 2048 x 2048
Allocating device variables...0.166348 s
Copying data from host to device...0.001513 s
Launching kernel...0.015063 s
Copying data from device to host...0.002665 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.061672 s
A: 2200 x 2200
B: 2200 x 2200
C: 2200 x 2200
Allocating device variables...0.141106 s
Copying data from host to device...0.001720 s
Launching kernel...0.015716 s
Copying data from device to host...0.003205 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.119126 s
A: 2048 x 3072
B: 3072 x 4096
C: 2048 x 4096
Allocating device variables...0.138379 s
Copying data from host to device...0.003268 s
Launching kernel...0.019772 s
Copying data from device to host...0.004680 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.119692 s
A: 2050 x 3080
B: 3080 x 4100
C: 2050 x 4100
Allocating device variables...0.139682 s
Copying data from host to device...0.003276 s
Launching kernel...0.019871 s
Copying data from device to host...0.004461 s
```

Verifying results...TEST PASSED

Setting up the problem...0.000005 s
A: 10 x 10
B: 10 x 10
C: 10 x 10
Allocating device variables...0.139191 s
Copying data from host to device...0.000032 s
Launching kernel...0.012792 s
Copying data from device to host...0.000013 s
Verifying results...TEST PASSED

Setting up the problem...0.000153 s
A: 100 x 100
B: 100 x 100
C: 100 x 100
Allocating device variables...0.136235 s
Copying data from host to device...0.000042 s
Launching kernel...0.012439 s
Copying data from device to host...0.000034 s
Verifying results...TEST PASSED

Setting up the problem...0.013623 s
A: 1000 x 1000
B: 1000 x 1000
C: 1000 x 1000
Allocating device variables...0.139641 s
Copying data from host to device...0.000455 s
Launching kernel...0.012881 s
Copying data from device to host...0.001192 s
Verifying results...TEST PASSED

Setting up the problem...1.256493 s
A: 10000 x 10000
B: 10000 x 10000
C: 10000 x 10000
Allocating device variables...0.137924 s
Copying data from host to device...0.032808 s
Launching kernel...0.568787 s
Copying data from device to host...0.043844 s
Matrices too large for CPU verification

Setting up the problem...5.030222 s
A: 20000 x 20000
B: 20000 x 20000
C: 20000 x 20000
Allocating device variables...0.159053 s
Copying data from host to device...0.130660 s
Launching kernel...4.562065 s

Copying data from device to host...0.189749 s
Matrices too large for CPU verification

Setting up the problem...0.000027 s
A: 10 x 60
B: 60 x 40
C: 10 x 40
Allocating device variables...0.142515 s
Copying data from host to device...0.000034 s
Launching kernel...0.012806 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED

Setting up the problem...0.002218 s
A: 100 x 600
B: 600 x 400
C: 100 x 400
Allocating device variables...0.141436 s
Copying data from host to device...0.000145 s
Launching kernel...0.012525 s
Copying data from device to host...0.000091 s
Verifying results...TEST PASSED

Setting up the problem...0.189132 s
A: 1000 x 6000
B: 6000 x 4000
C: 1000 x 4000
Allocating device variables...0.137041 s
Copying data from host to device...0.005056 s
Launching kernel...0.018923 s
Copying data from device to host...0.003056 s
Matrices too large for CPU verification

Setting up the problem...0.755323 s
A: 2000 x 12000
B: 12000 x 8000
C: 2000 x 8000
Allocating device variables...0.137447 s
Copying data from host to device...0.019803 s
Launching kernel...0.082758 s
Copying data from device to host...0.007844 s
Matrices too large for CPU verification

Setting up the problem...3.013994 s
A: 4000 x 24000
B: 24000 x 16000
C: 4000 x 16000
Allocating device variables...0.158583 s
Copying data from host to device...0.078563 s

```
Launching kernel...0.882995 s
Copying data from device to host...0.028658 s
Matrices too large for CPU verification
```

2 Analysis of the Performance

The first two outputs compare square matrices with sizes that fit and do not fit neatly in the blocks, respectively. The sizes in the second output are slightly larger, so it makes sense that copying data from host to device and vice versa takes slightly longer, as copying from one memory to another takes a long time. Even though the increase in matrix size was small, the kernel execution time also increased appreciably. For the second output, the matrix sizes are not evenly divisible by the block size (16x16). Because of this, divergence occurs in some of the warps, which results in longer execution time. Even though the matrices are not much larger, the extra time from divergence in the warps is enough to make the execution time noticeably longer. These same matrix sizes were also used in Lab 2, but here the kernel execution time was about 0.002 s faster due to tiling. Tiling reduced the number of global memory accesses and instead allowed the threads to access most of the values they needed from shared memory. Each thread only has to load a few elements from global memory, and can access the rest of the elements from shared memory because the other threads worked together to load those elements. This reduces the latency and bottlenecks associated with global memory accesses.

The third and fourth outputs compare non square matrices with sizes that fit and do not fit neatly in the blocks, respectively. Again, the sizes in the fourth output are slightly larger than the third, so it makes sense that copying data from host to device and vice versa takes slightly longer, as copying from one memory to another takes a long time. Also, even though the increase in matrix size between the third and fourth outputs was small, the kernel execution time increased appreciably. Just as with the square matrices, the matrix sizes for the fourth output were not evenly divisible by the block size. The resulting divergence in the warps took enough extra time to make the execution time noticeably longer than the third output, even though the matrices were only slightly larger. These same matrix sizes were also used in Lab 2, but here the kernel execution time was about 0.003 s faster due to tiling.

Outputs five through nine show times for square matrices of increasing size. The times for copying data from host to device and vice versa scale as the matrix sizes increase. This is expected, as copying memory from one location to another is mostly serial, so the time for this increases linearly with the data size. The kernel execution times do not increase noticeably, however, until the matrix sizes reach 10000 x 10000 x 10000. A possible explanation for this is a limitation in hardware resources. For matrix sizes below 10000 square, there are enough resources to process all the threads simultaneously. However, as the matrices increase in size, some of the calculations have to be serialized, as there are not enough resources to process all of the elements in parallel.

Outputs ten through fourteen show times for non square matrices of increasing size. Again, the times for copying data scale as the matrix sizes increase, which is expected. Also, the kernel execution times do not change noticeably between each other or from the square matrices, except for the last three cases. There are significant jumps in kernel execution times for the last three outputs. As with the square matrices, a reason for the large increase in kernel execution times could be hardware limitations. The GPU itself may have been able to execute all of the threads simultaneously for the smaller matrices. However, for the larger matrices, the GPU may not have been able to do all the threads at once, so it had to serialize the execution by a small amount.

3 Answer Section

1. In your kernel implementation, how many threads can be simultaneously scheduled for execution on a GeForce GTX 280 GPU, which contains 30 streaming multiprocessors? Use:

```
nvcc --ptxas-options="-v" kernel.cu
```

to see the resource usage of your kernel (although compilation will fail, it will only do so after compiling the kernel and displaying the relevant information). Show your work.

Listed below is the output of the nvcc command above:

```
ptxas info      : 0 bytes gmem
ptxas info      : Compiling entry function '_Z7mysgemmiiiPKfS0_Pf' for 'sm_52'
ptxas info      : Function properties for _Z7mysgemmiiiPKfS0_Pf
0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 30 registers, used 1 barriers, 2048 bytes smem, 360 bytes cmem[0]
ptxas info      : Compile time = 7.183 ms
```

Also listed below is the output from Lab 0 of the hardware resources available on the NVIDIA L40S GPU available on the "instructional" queue:

```
Device 0: "NVIDIA L40S"
Major revision number:      8
Minor revision number:      9
Total amount of global memory: 432537600 bytes
Number of multiprocessors:  142
Number of cores:            1136
Total amount of constant memory: 65536 bytes
Total amount of shared memory per block: 49152 bytes
Total number of registers available per block: 65536
Warp size:                  32
Maximum number of threads per SM: 1536
Maximum number of threads per block: 1024
Maximum sizes of each dimension of a block: 1024 x 1024 x 64
Maximum sizes of each dimension of a grid: 2147483647 x 65535 x 65535
Maximum memory pitch:       2147483647 bytes
Texture alignment:          512 bytes
Clock rate:                 2.52 GHz
Concurrent copy and execution: Yes
```

From the output of hardware resources, the maximum number of threads per SM is 1536. We are using blocks of size 16 threads x 16 threads for a total of 256 threads/block. The maximum number of blocks per SM is then:

$$\frac{\frac{1536 \text{ threads}}{SM}}{\frac{256 \text{ threads}}{\text{block}}} = \frac{1536 \text{ threads}}{SM} * \frac{\text{block}}{256 \text{ threads}} = 6 \text{ blocks}/SM$$

From the output of hardware resources, there are a total of 142 SM's, which means the maximum number of blocks with 256 threads/block that can run simultaneously is:

$$\frac{6 \text{ blocks}}{SM} * 142 \text{ SM's} = 852 \text{ blocks}$$

From the ptxas info above, the most used resource in the kernel is shared memory. Each thread is allocated 2048 bytes of shared memory. From the hardware resources, the total amount of shared

memory available per block is 49152 bytes. With 852 blocks, the total amount of used shared memory is then:

$$\frac{49152 \text{ bytes}}{\text{block}} * 852 \text{ blocks} = 41877504 \text{ bytes}$$

Since each thread is allocated 2048 bytes of shared memory, the total number of threads that can be simultaneously scheduled for execution on the L40S GPU available on "instructional" is then:

$$41877504 \text{ bytes} * \frac{\text{thread}}{2048 \text{ bytes}} = 20448 \text{ threads}$$